



CS2009

DESIGN AND ANALYSIS OF ALGORITHMS

Dr. Hashim Yasin

**National University of Computer
and Emerging Sciences,
Faisalabad, Pakistan.**

ALGORITHM



Algorithm

3

- An algorithm is a well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.
- We can also view an algorithm as a tool for solving a well-specified **computational problem**.
- The algorithm describes a specific computational procedure for achieving input/output relationship.
- Algorithm is finite number of **computational statements** that transform input into the output

Algorithm

4

- An algorithm is a well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.

For Example:

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Algorithm

5

Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

For example,

- ▣ given the input sequence (31; 41; 59; 26; 41; 58),
- ▣ a sorting algorithm
- ▣ returns as output sequence (26; 31; 41; 41; 58; 59).
- Such an input sequence is called an ***instance*** of the sorting problem

Evaluation of Algorithm

6

- **Completeness:** is the strategy guaranteed to find a solution when there is one?
- **Optimality:** does the strategy find the highest-quality (least-cost) solution when there are several different solutions?
- **Time complexity:** how long does it take to find a solution?
- **Space complexity:** how much memory is needed to perform the search?

Analysis of an Algorithm

7

- What is the goal of analysis of algorithms?
 - ▣ To compare algorithms mainly **in terms of running time** but also in terms of **other factors** (e.g., memory requirements, programmer's effort etc.)
- What does it mean by **running time analysis**?
 - ▣ **Determine how running time increases as the **size** of the problem increases.**

Analysis of an Algorithm

8

- ❑ Constant Time
- ❑ Logarithmic Time
- ❑ Linear Time
- ❑ Quadratic Time
- ❑ Polynomial Time
- ❑ Exponential Time

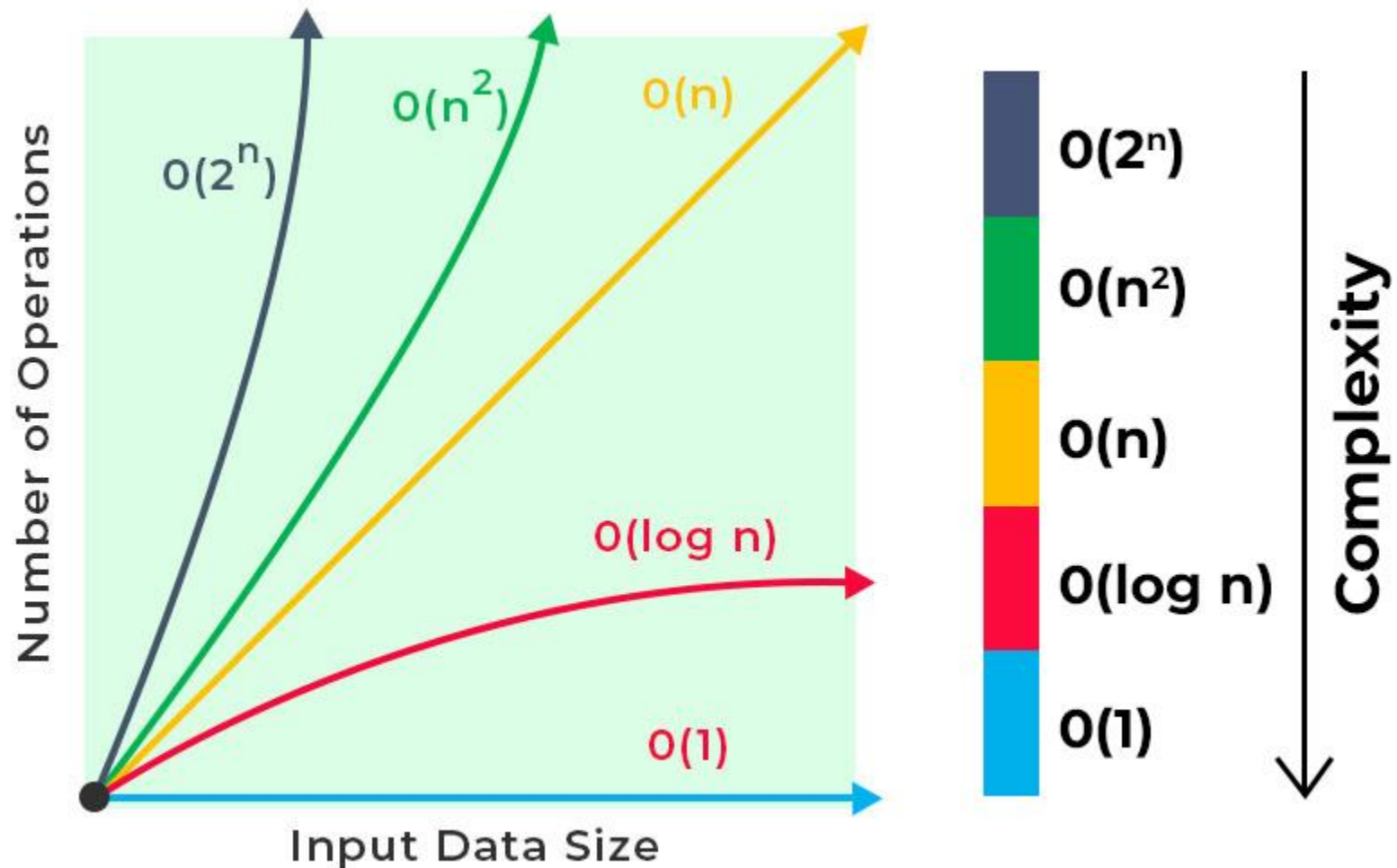
Analysis of an Algorithm

9

Complexity	Growth Rate	
$O(k) = O(1)$	Constant Time	 Increasing Complexity
$O(\log_b N) = O(\log N)$	Logarithmic Time	
$O(N)$	Linear Time	
$O(N \log N)$		
$O(N^2)$	Quadratic Time	
$O(N^p)$	Polynomial Time	
....		
$O(k^N)$	Exponential Time	

Comparative Analysis

10



TYPES OF ANALYSIS



Types of Analysis

12

□ Worst case

- ▣ Provides an upper bound on running time
- ▣ An absolute **guarantee** that the **algorithm will not run longer**, no matter what the inputs are.

□ Best case

- ▣ Provides a lower bound on running time
- ▣ Input is the one for which the **algorithm runs the fastest**

Types of Analysis

13

□ Average case

- ▣ Provides a **prediction** about the running time
- ▣ Assumes that the input is random.

$$\textit{Lower Bound} \leq \textit{Running Time} \leq \textit{Upper Bound}$$

Types of Analysis

14

□ Worse Case:

▣ For example, when we are searching for number 23

31	47	15	78	39	17	23
----	----	----	----	----	----	----

□ Best Case:

▣ For example, when we are searching for number 31

31	47	15	78	39	17	23
----	----	----	----	----	----	----

□ Average Case:

▣ For example, when we are searching for any number,

31	47	15	78	39	17	23
----	----	----	----	----	----	----

ASYMPTOTIC NOTATIONS



Asymptotic Notation

16

- Asymptotic notations – Asymptotic notations are the notations that are used to describe **the behavior of the time or space complexity**.
- Let us represent the time complexity and the space complexity using the common function $f(n)$.
 - O (Big Oh notation)
 - Ω (Omega notation)
 - θ (Theta notation)
 - o (Little Oh notation)
 - ω (Little Omega notation)

O – Big Oh notation

17

- The big Oh notation provides an **upper bound** (a **Worst Case of an algorithm**) for the function $f(n)$.
- The function $f(n) = O(g(n))$ if and only if there exists positive constants c and n_0 such that

$$0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0$$

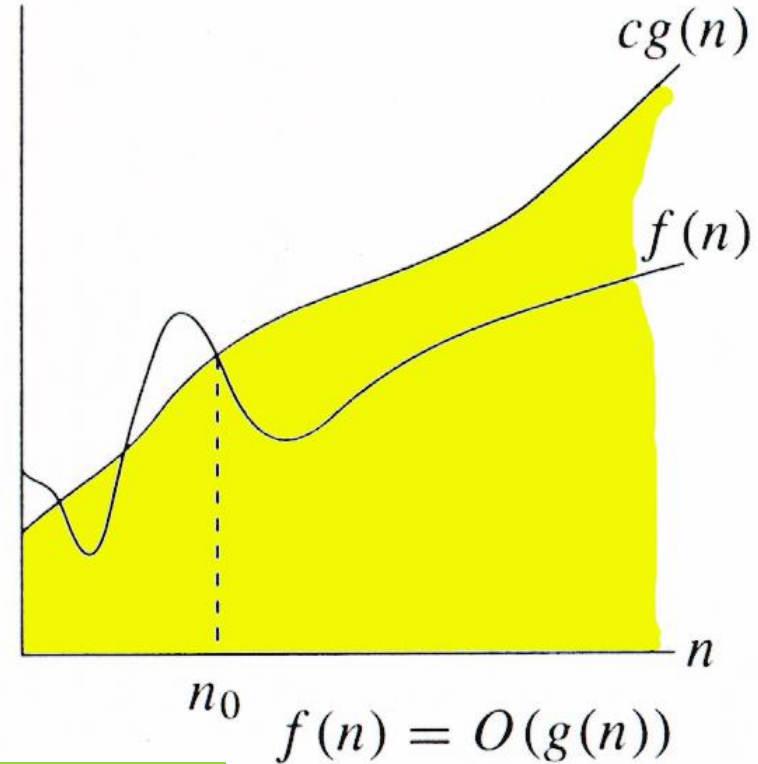
O – Big Oh notation

18

The function $f(n) = O(g(n))$
if and only if there exists positive
constants c and n_0 such that

$$0 \leq f(n) \leq cg(n)$$

for all $n \geq n_0$



$g(n)$ is an **asymptotic upper bound** for $f(n)$.

O – Big Oh notation

19

$$f(n) = 3n + 2$$

$$0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0$$

Let us take $g(n) = n$

$$c = 4$$

$$n_0 = 2$$

Let us check the above condition

$$3n + 2 \leq 4n \quad \text{for all } n \geq 2$$

The condition is satisfied. Hence $f(n) = O(n)$.

O – Big Oh notation

20

$$f(n) = 10n^2 + 4n + 2 \quad 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0$$

Let us take $g(n) = n^2$

$$c = 11$$

$$n_0 = 5$$

Let us check the above condition

$$10n^2 + 4n + 2 \leq 11n^2 \quad \text{for all } n \geq 5$$

The condition is satisfied. Hence $f(n) = O(n^2)$.

Ω - Omega notation

21

- The omega notation (Ω) provides a **lower bound** for the function $f(n)$.
- The function $f(n) = \Omega(g(n))$ if and only if there exists positive constants c and n_0 such that

$$f(n) \geq cg(n) \geq 0 \text{ for all } n \geq n_0$$

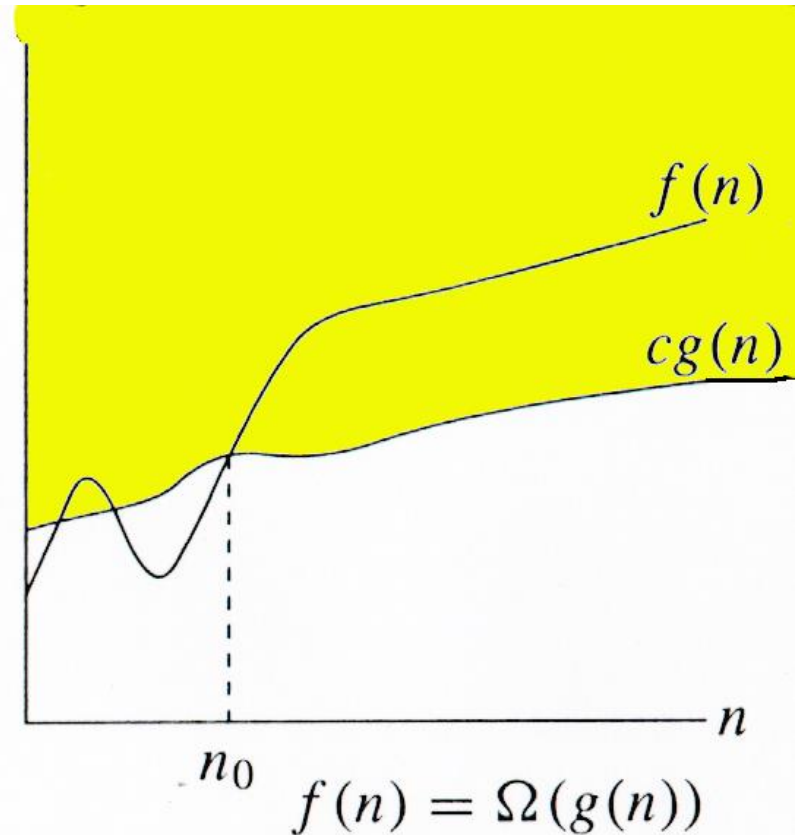
Ω - Omega notation

22

The function $f(n) = \Omega(g(n))$
if and only if there exists
positive constants c and n_0
such that

$$f(n) \geq cg(n) \geq 0$$

for all $n \geq n_0$



$g(n)$ is an **asymptotic lower bound** for $f(n)$.

θ – Theta notation

23

- The theta notation (θ) is used when the function $f(n)$ can be bounded by both from above and below the same function $g(n)$.
- The function $f(n) = \theta(g(n))$ if and only if there exists positive constants c_1, c_2 and n_0 such that

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0$$

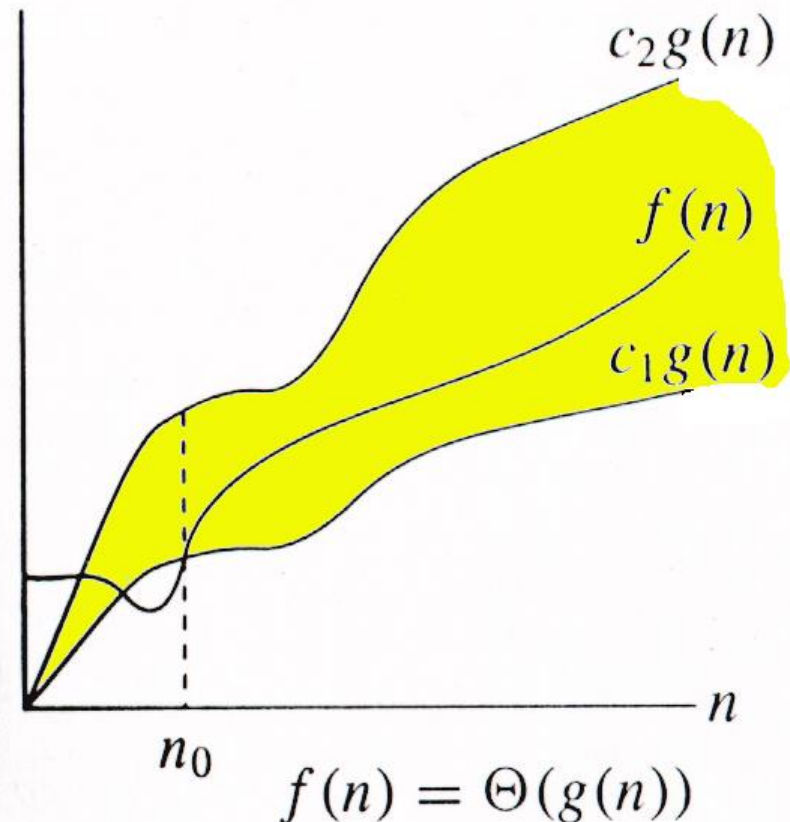
θ – Theta notation

24

The function $f(n) = \theta(g(n))$
if and only if there exists positive
constants c_1, c_2 and n_0 such that

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

for all $n \geq n_0$



$g(n)$ is an asymptotically tight bound for $f(n)$.

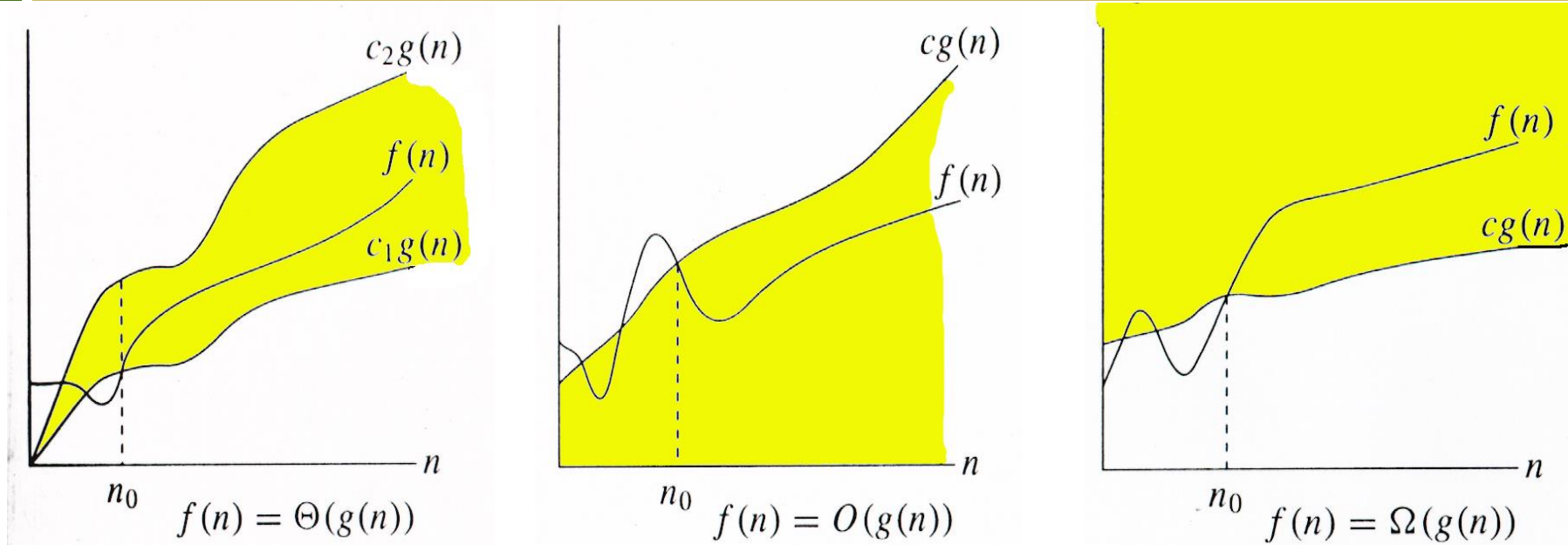
Asymptotic Notation

25

- O notation: asymptotic “less than”:
 - ▣ $f(n) = O(g(n))$ implies: $f(n) \leq g(n)$
- Ω notation: asymptotic “greater than”:
 - ▣ $f(n) = \Omega(g(n))$ implies: $f(n) \geq g(n)$
- θ notation: asymptotic “equality”:
 - ▣ $f(n) = \theta(g(n))$ implies: $f(n) = g(n)$

Relationship (O , Ω , and θ)

26



Theorem : For any two functions $g(n)$ and $f(n)$,
 $f(n) = \theta(g(n))$ iff
 $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Relationship (O , Ω , and θ)

27

Theorem : For any two functions $g(n)$ and $f(n)$,
 $f(n) = \theta(g(n))$ iff
 $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

- i.e., $\theta(g(n)) = O(g(n)) \cap \Omega(g(n))$
- In practice, asymptotically tight bounds are obtained from asymptotic upper and lower bounds.

o - Little Oh notation

28

- The little o notation (o) is,
- The function $f(n) = o(g(n))$ if and only if there exists positive constants c and n_0 such that

$$0 \leq f(n) < cg(n) \text{ for all } n \geq n_0$$

- $g(n)$ is an **upper bound** for $f(n)$ that is **NOT asymptotically tight**.

o - Little Oh notation

29

□ The little *o* notation (*o*) is,

$f(n) = o(g(n))$ if and only if

$f(n) = O(g(n))$ and

$f(n) \neq \Omega(g(n))$

o - Little Oh notation

30

The definitions of O -notation and o -notation are similar. The main difference is that in $f(n) = O(g(n))$, the bound $0 \leq f(n) \leq cg(n)$ holds for *some* constant $c > 0$, but in $f(n) = o(g(n))$, the bound $0 \leq f(n) < cg(n)$ holds for *all* constants $c > 0$. Intuitively, in o -notation, the function $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity

ω - Little Omega notation

31

- The little ω notation (ω) is,
- The function $f(n) = \omega(g(n))$ if and only if there exists positive constants c and n_0 such that

$$0 \leq cg(n) < f(n) \text{ for all } n \geq n_0$$

- $g(n)$ is a **lower bound** for $f(n)$ that is **NOT asymptotically tight.**

PROPERTIES

1. Transitivity

33

Transitivity:

$f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$ imply $f(n) = \Theta(h(n))$,
 $f(n) = O(g(n))$ and $g(n) = O(h(n))$ imply $f(n) = O(h(n))$,
 $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ imply $f(n) = \Omega(h(n))$,
 $f(n) = o(g(n))$ and $g(n) = o(h(n))$ imply $f(n) = o(h(n))$,
 $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$ imply $f(n) = \omega(h(n))$.

2. Reflexivity

34

Reflexivity:

$$f(n) = \Theta(f(n)) ,$$

$$f(n) = O(f(n)) ,$$

$$f(n) = \Omega(f(n)) .$$

3. Symmetry

35

Symmetry:

$f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$.

Transpose symmetry:

$f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$,

$f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$.

Comparing Functions

36

Because these properties hold for asymptotic notations, we can draw an analogy between the asymptotic comparison of two functions f and g and the comparison two real numbers a and b :

$f(n) = O(g(n))$ is like $a \leq b$,

$f(n) = \Omega(g(n))$ is like $a \geq b$,

$f(n) = \Theta(g(n))$ is like $a = b$,

$f(n) = o(g(n))$ is like $a < b$,

$f(n) = \omega(g(n))$ is like $a > b$.

Typical Big O Function – "Grades"

Function	Common Name
$N!$	Factorial
2^N	Exponential
$N^d, d > 3$	Polynomial
N^3	Cubic
N^2	Quadratic
$N\sqrt{N}$	N Square root N
$N \log N$	$N \log N$
N	Linear
$\sqrt{N}$	Root - n
$\log N$	Logarithmic
1	Constant



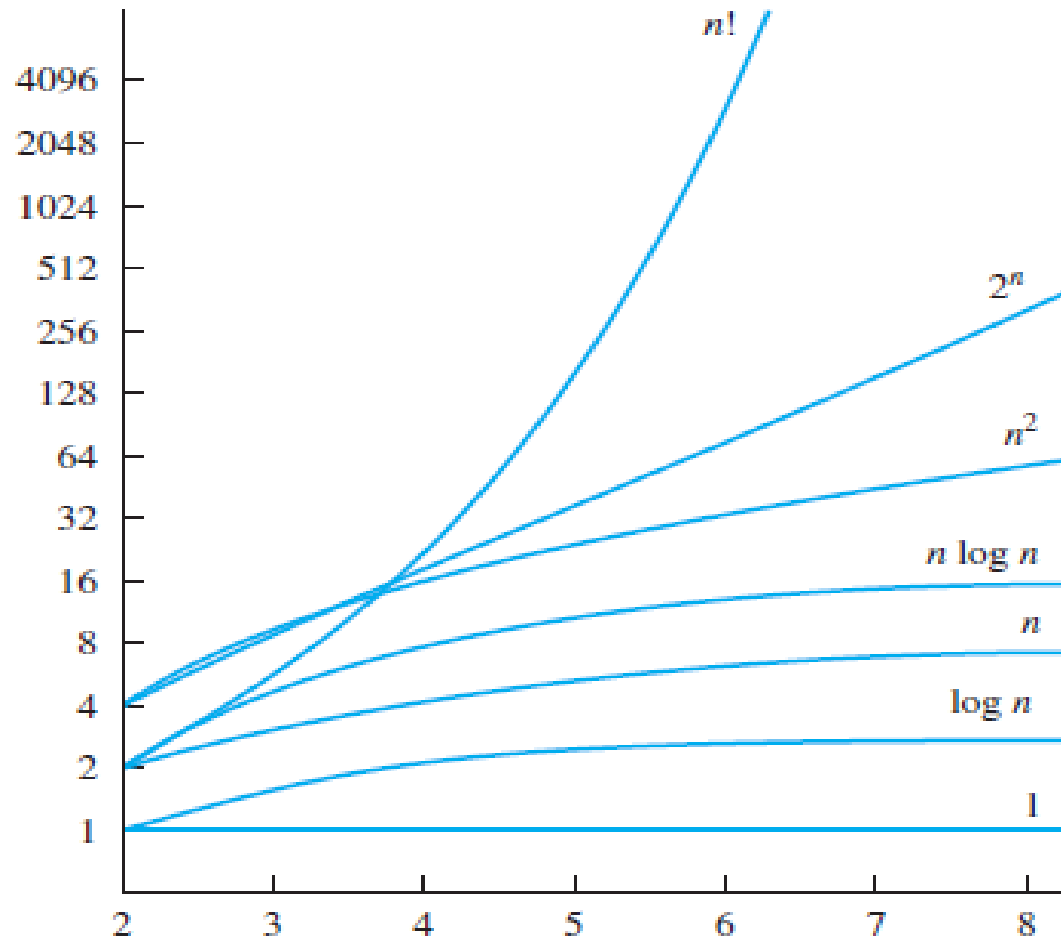
Running time grows 'quickly' with more input.



Running time grows 'slowly' with more input.

Growth of Function (Big O)

38



Growth of Function (Big O)

39

Time for $f(n)$ instructions on a computer that executes 1 billion instructions per second

n	$f(n) = n$	$f(n) = \log_2 n$	$f(n) = n \log_2 n$	$f(n) = n^2$	$f(n) = 2^n$
10	0.01 μ s	0.003 μ s	0.033 μ s	0.1 μ s	1 μ s
20	0.02 μ s	0.004 μ s	0.086 μ s	0.4 μ s	1ms
30	0.03 μ s	0.005 μ s	0.147 μ s	0.9 μ s	1s
40	0.04 μ s	0.005 μ s	0.213 μ s	1.6 μ s	18.3min
50	0.05 μ s	0.006 μ s	0.282 μ s	2.5 μ s	13 days
100	0.10 μ s	0.007 μ s	0.664 μ s	10 μ s	4×10^{13} years
1000	1.00 μ s	0.010 μ s	9.966 μ s	1ms	
10,000	10 μ s	0.013 μ s	130 μ s	100ms	
100,000	0.10ms	0.017 μ s	1.67ms	10s	
1,000,000	1 ms	0.020 μ s	19.93ms	16.7m	
10,000,000	0.01s	0.023 μ s	0.23s	1.16 days	
100,000,000	0.10s	0.027 μ s	2.66s	115.7 days	

Growth of Function (Big O)

40

□ It can be shown that,

$$O(1) \leq O(\log_2 n) \leq O(n) \leq O(n \log_2 n) \leq O(n^2) \leq O(2^n).$$

n dominates $\log_2(n)$

$(n \times \log_2(n))$ dominates n

n^2 dominates $(n \times \log_2(n))$

n^m dominates n^k when $m > k$

a^n dominates n^m for any $a > 1$ and $m \geq 0$

Acknowledgement

41

Introduction to Algorithms

Thomas H. Cormen, Charles E. Leiserson,
Ronald L. Rivest, and Clifford Stein